



node pack

Sick Ollie Node Pack Field Guide

Updated reference edition for Loader, Prompt, Generation, Output, Preview, and resolved-prompt persistence.

What's New in v1.1.0

Version 1.1.0 is a substantial production update. It expands the two most automation-heavy nodes, strengthens workflow persistence, and gives the pack a cleaner way to move LoRA identity information into prompts and filenames.

NEW

Loader Core: epoch-aware testing

Filter a folder by detected epoch numbers, choose cleaner identity names from a dropdown, keep up to ten secondary LoRAs persistent across workflow reloads, and expose the selected main LoRA trigger when metadata provides one.

NEW

Prompt Core: three independent outfit channels

Use OUTFIT_A, OUTFIT_B, and OUTFIT_C in the same prompt, each driven by its own log, mode, and index. Prompt, outfit, and scene logs loop automatically and skip blank lines.

FIXED

Stable prompt state

Prompt settings, cleanup rules, indexes, prefix and suffix text, and the resolved prompt display now survive tab changes, workflow reopening, and image-based workflow imports.

FIXED

Output and Preview persistence

Output Core keeps its latest resolved values and saved path visible. Preview Core restores its preview image when a workflow is reconstructed.

NEW

Immediate trigger reference

The Loader copy button updates as soon as a main LoRA is selected. The main_trigger socket emits only while the main LoRA is active, preventing disabled LoRAs from injecting stale triggers into connected prompt fields.

Release scope: This guide describes the final public v1.1.0 behavior. Temporary development-only problems and private test cases are intentionally omitted.

Contents

System Overview.....	4
The Six Nodes.....	4
Core Philosophy.....	4
Installation and Starter Content.....	6
Quick-Start Wiring.....	6
Chapter 1: Loader Core.....	7
Folder and Epoch Cycling.....	7
Name Outputs and Clean Names.....	8
Trigger Detection.....	9
Persistent Secondary LoRAs.....	9
Controls and Outputs.....	10
Chapter 2: Prompt Core.....	11
Logs and Progression.....	11
OUTFIT_A, B, and C.....	12
Prefix, Suffix, and Cleanup.....	13
Controls and Outputs.....	14
Chapter 3: Generation Core.....	16
Controls and Seed System.....	16
Chapter 4: Output Core.....	18
Folder and Filename Builders.....	18
Metadata and Civitai Resources.....	19
Controls and Outputs.....	20
Chapter 5: Preview Core.....	21
Fit and Background Modes.....	21
Chapter 6: Persist + Display Resolved Prompt.....	24
Complete Wiring Guide.....	25
Workflow Recipes.....	27
Troubleshooting.....	29
Glossary.....	32
Compatibility, Credits, and License.....	32

System Overview

The Six Nodes

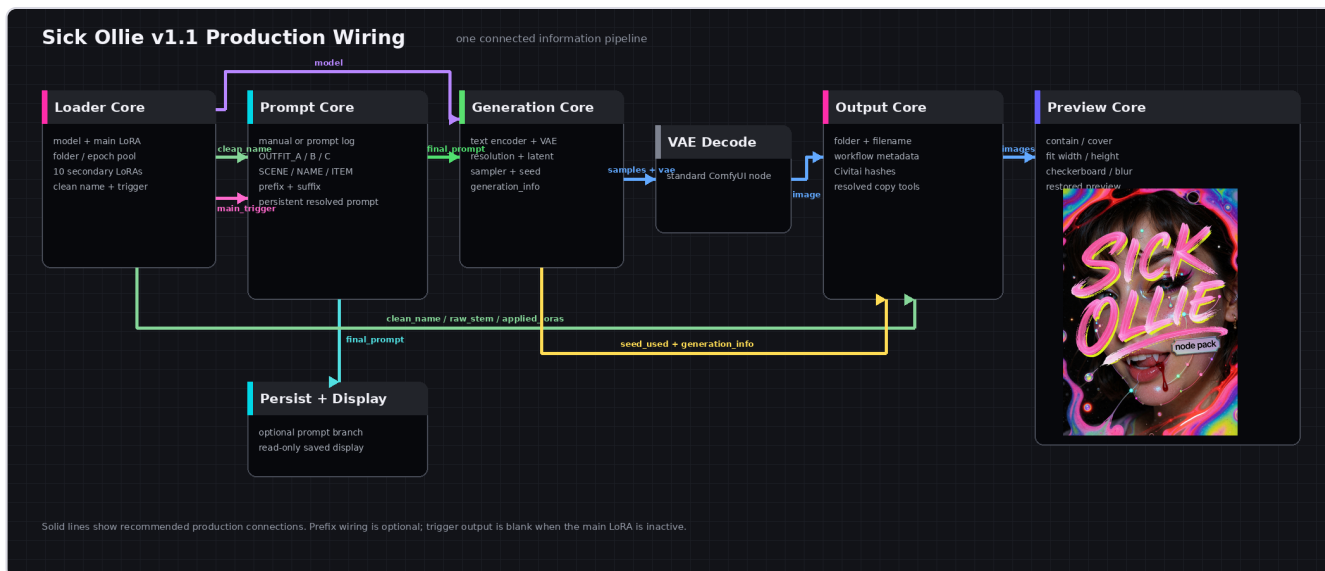
Node	Primary role
Loader Core	Loads the diffusion model, selects a folder-scoped main LoRA, applies persistent secondary LoRAs, and produces identity, file, and trigger strings.
Prompt Core	Builds a final prompt from manual text or logs, replaces NAME, ITEM, SCENE, and three independent outfit tokens, then applies prefix, suffix, and cleanup.
Generation Core	Loads the text encoder and VAE, encodes the positive prompt, creates the latent, samples the image, and returns the exact seed and generation data.
Output Core	Builds portable folders and filenames, saves the image, embeds workflows and prompt data, hashes resources, and writes Civitai-readable parameters.
Preview Core	Displays images using fit and background modes while passing the original tensor through unchanged.
Persist + Display Resolved Prompt	Optional text passthrough that keeps a resolved prompt readable inside saved workflows.

Core Philosophy

The pack follows one rule: create information once, transform it intentionally, and carry it forward. A selected LoRA already contains enough context to produce its exact file path, filename stem, cleaned identity name, applied-stack record, and sometimes a trigger. Those outputs can feed prompt construction, naming, metadata, and later inspection without manually retyping the same facts in several places.

```
main_file    = exact selected LoRA path and filename
raw_stem     = exact filename without folder or extension
clean_name   = human-friendly identity or style name
main_trigger = detected trigger while the main LoRA is active
```

Practical rule: Use `clean_name` when you mean “who or what is this?” Use `raw_stem` when you mean “which exact training file produced this?”



The v1.1 production chain. Loader Core feeds model and identity data, Prompt Core resolves the final text, Generation Core renders, Output Core saves, and Preview Core displays.

Installation and Starter Content

Requirements

- ComfyUI
- rgthree-comfy

rgthree-comfy supplies the enhanced secondary-LoRA chooser, row drawing, information dialog, and related interface utilities used by Loader Core.

Install the pack

1. Extract the release archive.
2. Copy **ComfyUI-SickOllie** into **ComfyUI/custom_nodes/**.
3. Confirm **rgthree-comfy** is installed beside it.
4. Restart ComfyUI completely.
5. Hard-refresh the browser with **Ctrl+F5**.

```
ComfyUI/
├─ custom_nodes/
│   └─ ComfyUI-SickOllie/
│       └─ rgthree-comfy/
```

Starter content

The pack includes a ready-to-open workflow and prompt-log folders beneath **Starter Content**. Copy the included **SickOllieLogs** folder into the ComfyUI input directory:

```
ComfyUI/input/SickOllieLogs/
├─ prompts/
├─ outfits/
└─ scenes/
```

Restart and hard-refresh after adding new folders so Prompt Core rebuilds its dropdowns.

Quick-Start Wiring

```
Loader Core model → Generation Core model
Loader Core clean_name → Prompt Core name_value
Loader Core main_trigger → Prompt Core prefix_text (optional)
Loader Core clean_name → Output Core clean_name
Loader Core raw_stem → Output Core raw_stem
Loader Core diffusion_model_stem → Output Core model_name
Loader Core applied_loras → Output Core applied_loras
Prompt Core final_prompt → Generation Core positive_text
Generation Core samples → VAE Decode samples
Generation Core vae → VAE Decode vae
Generation Core seed_used → Output Core seed
Generation Core generation_info → Output Core generation_info
VAE Decode IMAGE → Output Core images
Output Core images → Preview Core images
```

Chapter 1: Loader Core

Loader Core is the front gate of the workflow. It loads the diffusion model, filters and cycles a primary LoRA, applies up to ten persistent secondary LoRAs, and exposes strings that downstream nodes can reuse.



Loader Core with a selected primary LoRA, detected trigger, automatic clean-name choice, and four persistent secondary styles.

Folder-Scoped and Epoch-Scoped LoRA Cycling

The folder dropdown is generated from the real ComfyUI LoRA directory. Selecting a folder seals the main-LoRA pool to that location. `include_subfolders` decides whether nested folders join the pool.

The epoch filter adds a second gate. It recognizes common forms such as `epoch1`, `epoch_1`, `epoch-01`, and `epoch 001`, then presents canonical choices such as `Epoch 1`. `[No epoch tag]` isolates files without an epoch marker.



The epoch filter is built from the files available inside the selected folder scope.

Progression modes

Mode	Behavior
fixed	Keeps the selected main LoRA unchanged after queuing.
increment	Advances to the next LoRA after each prompt is queued.
decrement	Moves backward through the active pool.
randomize	Chooses a different eligible LoRA when possible.

`loop_folder` wraps from the end to the beginning. `skip_none_during_cycle` leaves None available for manual use but removes it from automatic progression.

Queue-time behavior: Progression occurs when prompts are queued, not when each image finishes. A large Queue count can therefore assign a different LoRA to each queued prompt before execution begins.

The Name System

Loader Core produces several versions of the selected resource name because each downstream job needs a different level of precision.

Output	Meaning	Typical use
main_file	Exact selected LoRA path and filename.	Forensic metadata or exact source identification.
raw_stem	Filename without folder and extension.	Versioned filenames and traceable output naming.
clean_name	Human-friendly name selected from automatic trim choices.	NAME replacement and tidy folders.

Output	Meaning	Typical use
diffusion_model_file	Exact diffusion-model filename.	Metadata and exact model tracking.
diffusion_model_stem	Diffusion-model filename without extension.	Readable model naming in Output Core.

Automatic clean-name choices

When `auto_clean_name` is enabled, the clean-name field becomes a real dropdown. Loader Core examines suffix groups shared by the active pool and recognized training markers, then offers progressively less-trimmed names.

```
example_identity_krea2_epoch_10.safetensors
```

Possible choices:

- 1 · example_identity
- 2 · example_identity_krea2
- 3 · example_identity_krea2_epoch_10

The selected trim level is applied consistently as the main LoRA changes, which is especially useful when cycling many epochs or versions.

Main-LoRA Trigger Detection

Loader Core reads the SafeTensor metadata header locally as soon as the main LoRA is selected. The copy button updates before generation, making the trigger visible while the workflow is still being assembled.

Detection priority

1. Explicit trigger fields, including `modelspec.trigger_phrase` and common trigger or trained-word keys.
2. The best candidate from `ss_tag_frequency`, led by frequency with light filtering of generic caption tags.
3. `modelspec.title`.
4. `none` when no usable trigger metadata exists.

Active-output safety: The copy button may show the selected file's detected trigger for reference. The `main_trigger` output remains blank when `main_enabled` is off, the main file is None, or main strength is zero. A disabled LoRA therefore cannot inject its trigger into a connected prompt field.

Trigger detection is best effort. Some LoRAs are intentionally triggerless, and some trainers do not embed the caption or trigger into the exported file. Those LoRAs correctly show `none`.

Ten Persistent Secondary LoRAs

Loader Core contains a serialized pool of ten secondary slots. Empty slots stay hidden until needed. Clicking **Add Secondary LoRA** reveals and fills the next free slot.

- Each row stores enabled state, LoRA filename, and strength.
- Rows remain present after switching tabs, reopening workflows, or importing a workflow-bearing PNG.
- The header toggle enables or disables all populated rows.
- Right-click a row and choose **Clear Row** to empty it and return the slot to the hidden pool.
- The information button opens rgthree's LoRA information dialog.

Separation of roles: Secondary LoRAs do not participate in folder or epoch cycling. They remain fixed while the main LoRA progresses.

Loader Core Controls

`diffusion_model`

Diffusion model loaded internally before any LoRA is applied.

`weight_dtype`

Model loading dtype: default, FP8 E4M3FN, fast FP8 E4M3FN, or FP8 E5M2.

`folder_name`

Restricts the main-LoRA dropdown and cycling pool to one folder.

`epoch_filter`

Restricts the active folder pool to a detected epoch or files with no epoch tag.

`main_enabled`

Enables or disables the primary LoRA.

`main_lora`

Primary LoRA selected manually or changed by progression.

`main_strength`

Weight applied to the primary LoRA.

`include_subfolders`

Adds nested folders beneath the selected folder.

`loop_folder`

Wraps automatic progression at the end of the pool.

`control_after_generate`

Fixed, increment, decrement, or randomize.

`skip_none_during_cycle`

Excludes None from automatic movement.

`off_name`

String used by name outputs while the primary LoRA is inactive.

`auto_clean_name`

Enables automatic clean-name selection.

`clean_name`

Dropdown of trim levels detected from the active pool.

`Copy trigger`

Copies the detected SafeTensor trigger immediately.

`secondary rows`

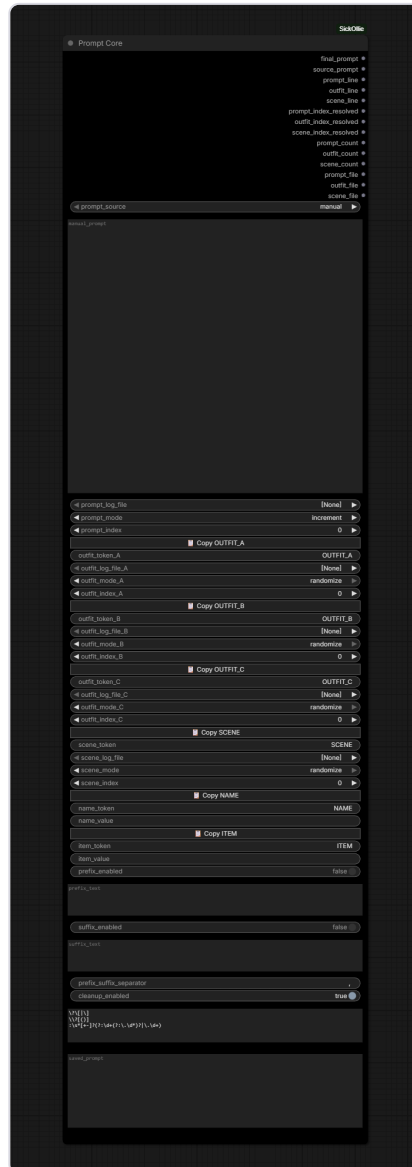
Up to ten persistent fixed LoRAs, each with toggle and strength.

Loader Core Outputs

Output	Description
<code>model</code>	Diffusion model with the active main and secondary LoRAs applied.
<code>diffusion_model_file</code>	Exact selected diffusion-model filename.
<code>diffusion_model_stem</code>	Diffusion-model filename without extension.
<code>main_file</code>	Exact active main-LoRA path and filename, or <code>off_name</code> when inactive.
<code>raw_stem</code>	Exact active main filename stem, or <code>off_name</code> when inactive.
<code>clean_name</code>	Selected human-friendly clean name.
<code>applied_loras</code>	Newline-separated list of LoRAs actually applied, including strengths.
<code>main_active</code>	True only when the main LoRA is enabled, selected, and nonzero strength.
<code>folder_count</code>	Number of main LoRAs inside the active folder and epoch pool.
<code>main_trigger</code>	Detected trigger while the main LoRA is active; otherwise blank.

Chapter 2: Prompt Core

Prompt Core combines a manual prompt or reusable prompt log with independent outfit logs, a scene log, literal tokens, optional prefix and suffix text, and cleanup rules. The resolved result remains visible after the workflow is reconstructed.



Prompt Core's v1.1 layout. The three outfit channels are native, independent controls rather than temporary UI additions.

Prompt Logs and Progression

A prompt log is a UTF-8 text file containing one reusable prompt per nonblank line. Outfit and scene logs use the same rule. Blank lines are skipped automatically, and every log loops by design.

```
ComfyUI/input/SickOllieLogs/
├─ prompts/
├─ outfits/
└─ scenes/
```

Modes

Mode	Behavior
fixed	Uses the selected raw index without changing it.
increment	Advances after each prompt is queued.
decrement	Moves backward after each queue action.
randomize	Chooses a valid line index for each queued prompt.

Indexes may become very large during long runs. The resolved index wraps into the usable line count, so the actual selected line remains valid.

Automatic looping: Prompt, outfit, and scene logs no longer expose loop switches. The pack always wraps instead of becoming stuck on the final line.

The Literal Token Pipeline

Token matching is literal and case-sensitive. Prompt Core replaces every exact occurrence of the configured token only when a usable replacement value exists.

```
Source prompt:
NAME stands in SCENE wearing OUTFIT_A beside a second subject in OUTFIT_B.

Resolved values:
NAME      → example_identity
SCENE     → a fluorescent roadside diner
OUTFIT_A  → a striped babydoll dress and knee-high socks
OUTFIT_B  → an oversized loose tank and color-block briefs
```

ITEM remains available for a manually supplied object, prop, product, or recurring motif.

OUTFIT_A, OUTFIT_B, and OUTFIT_C

Each outfit channel has its own token, file, mode, and index. A single prompt can therefore assign different clothing to as many as three subjects without paired-line delimiters.

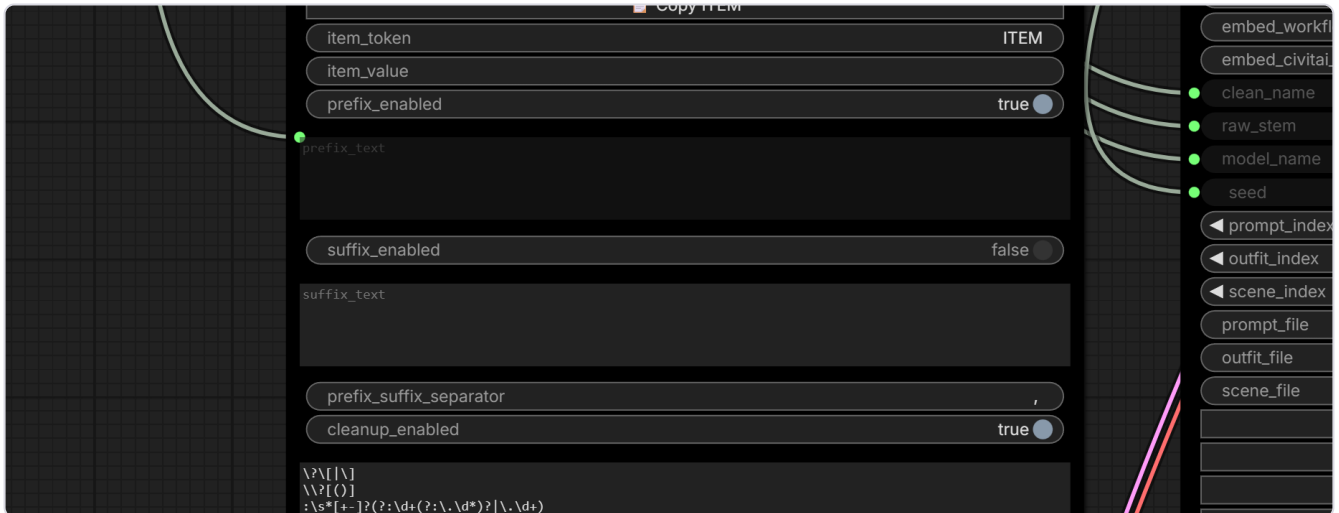
Channel	Default token	Independent controls
A	OUTFIT_A	outfit_log_file_A, outfit_mode_A, outfit_index_A
B	OUTFIT_B	outfit_log_file_B, outfit_mode_B, outfit_index_B
C	OUTFIT_C	outfit_log_file_C, outfit_mode_C, outfit_index_C

Primary outfit outputs: The public `outfit_line`, `outfit_index_resolved`, `outfit_count`, and `outfit_file` outputs represent channel A. When A has no usable log, they fall back to B and then C. All three resolved lines are still preserved inside workflow metadata.

Copy-token buttons

Buttons for OUTFIT_A, OUTFIT_B, OUTFIT_C, SCENE, NAME, and ITEM copy the current literal token. They do not modify the prompt or consume saved workflow-value positions.

Prefix, Suffix, and Cleanup



Prefix and suffix are separate controls. Each may also accept a connected string input.

`prefix_text` is joined before the assembled prompt when `prefix_enabled` is true. `suffix_text` is joined afterward when `suffix_enabled` is true. `prefix_suffix_separator` controls the join text, commonly , .

```
prefix_text: detected_trigger
assembled prompt: a candid portrait at a roadside diner
suffix_text: harsh direct flash, early-2000s color response
separator: ,

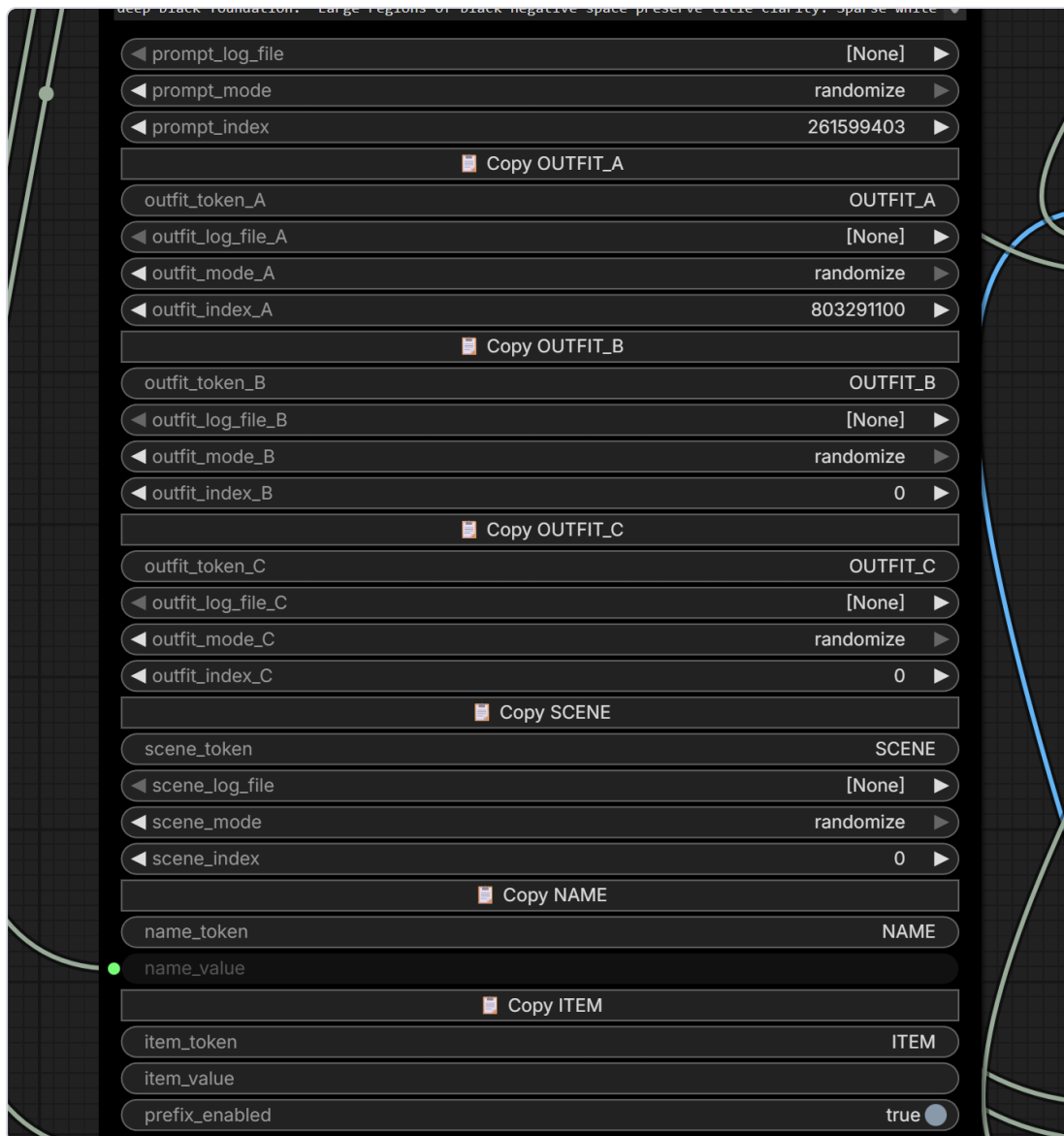
Final:
detected_trigger, a candid portrait at a roadside diner, harsh direct flash, early-2000s color response
```

Cleanup runs last. The default rules remove common prompt-weight punctuation and numeric weight suffixes while leaving ordinary prose intact.

Connected input note: A string connected to `prefix_text` or another widget-backed input replaces the local text value for execution. Disconnect the wire to resume editing the local field directly.

Resolved Prompt Persistence

The saved-prompt display is read-only production feedback. It records the most recently resolved final prompt inside node properties and workflow metadata. It therefore survives tab changes, workflow reopening, and workflow-bearing image imports.



The final resolved prompt remains visible without requiring a separate display node.

Prompt Core Controls

prompt_source

manual or log

manual_prompt

multiline manual source text

prompt_log_file

selected prompt text file

prompt_mode / prompt_index

line-selection behavior

outfit_token_A/B/C

literal tokens for the three outfit channels

outfit_log_file_A/B/C

independent outfit files

outfit_mode_A/B/C

independent progression modes

outfit_index_A/B/C

independent raw indexes

scene_token / scene_log_file

scene replacement channel

scene_mode / scene_index

scene progression

name_token / name_value

identity replacement, usually fed by Loader
clean_name

item_token / item_value

manual reusable object replacement

prefix_enabled / prefix_text

optional text before the assembled prompt

suffix_enabled / suffix_text

optional text after the assembled prompt

`prefix_suffix_separator`
join text between nonblank prompt parts

`cleanup_enabled` / `cleanup_rules`
final normalization pass

`saved_prompt`
read-only most recent final prompt

Prompt Core Outputs

Output	Description
<code>final_prompt</code>	Final resolved and cleaned prompt.
<code>source_prompt</code>	Manual prompt or selected prompt-log line before replacements.
<code>prompt_line</code>	Selected prompt-log line.
<code>outfit_line</code>	Primary outfit line from A, with B/C fallback when A is empty.
<code>scene_line</code>	Selected scene line.
<code>prompt_index_resolved</code>	Wrapped prompt index actually used.
<code>outfit_index_resolved</code>	Resolved index for the primary outfit output.
<code>scene_index_resolved</code>	Resolved scene index.
<code>prompt_count</code> / <code>outfit_count</code> / <code>scene_count</code>	Usable line counts for the corresponding primary channels.
<code>prompt_file</code> / <code>outfit_file</code> / <code>scene_file</code>	Relative selected log filenames.

Chapter 3: Generation Core

Generation Core keeps the text encoder, VAE, resolution, latent creation, sampling, live preview, and seed reporting inside one compact Krea2-oriented node.

Why it exists

A conventional graph may require separate text-encoder loading, positive and negative encoding, latent creation, model sampling, VAE loading, and several primitive nodes. Generation Core deliberately centralizes those steps while returning the latent and VAE separately for a normal VAE Decode.

```
Loader Core model → Generation Core model
Prompt Core final_prompt → Generation Core positive_text
Generation Core samples → VAE Decode samples
Generation Core vae → VAE Decode vae
```

Controls

Control	Purpose
<code>model</code>	LoRA-patched diffusion model from Loader Core.
<code>positive_text</code>	Resolved positive prompt. The node supplies fixed empty negative conditioning internally.
<code>clip_name</code>	Text-encoder file.
<code>clip_type</code>	Encoder architecture, including krea2.
<code>clip_device</code>	Default, CPU, or other supported encoder device behavior.
<code>vae_name</code>	VAE file loaded and returned.
<code>resolution_mode</code>	custom or preset.
<code>custom_width / custom_height</code>	Direct dimensions in custom mode.
<code>aspect_preset / megapixels</code>	Aspect and target area in preset mode.
<code>batch_size</code>	Latents generated per execution.
<code>steps / cfg</code>	Sampling controls.
<code>sampler_name / scheduler</code>	Comfy sampler and scheduler choices.
<code>denoise</code>	Sampling denoise amount.
<code>shift</code>	Model shift applied to the sampling model.
<code>seed_value</code>	Exact seed, or -1 for independent randomization.

The Seed System

-1 randomizes independently for each execution. The node stores and displays the actual seed used, and returns it through **seed_used**. The copy button always targets that resolved seed, not the placeholder **-1**.

- **Randomize Each Time** returns seed control to -1.
- **New Fixed Random** generates a new fixed seed immediately.
- **Use Last Queued Seed** writes the latest resolved seed back into the editable field.
- **Copy Last Used Seed** copies the exact value used most recently.

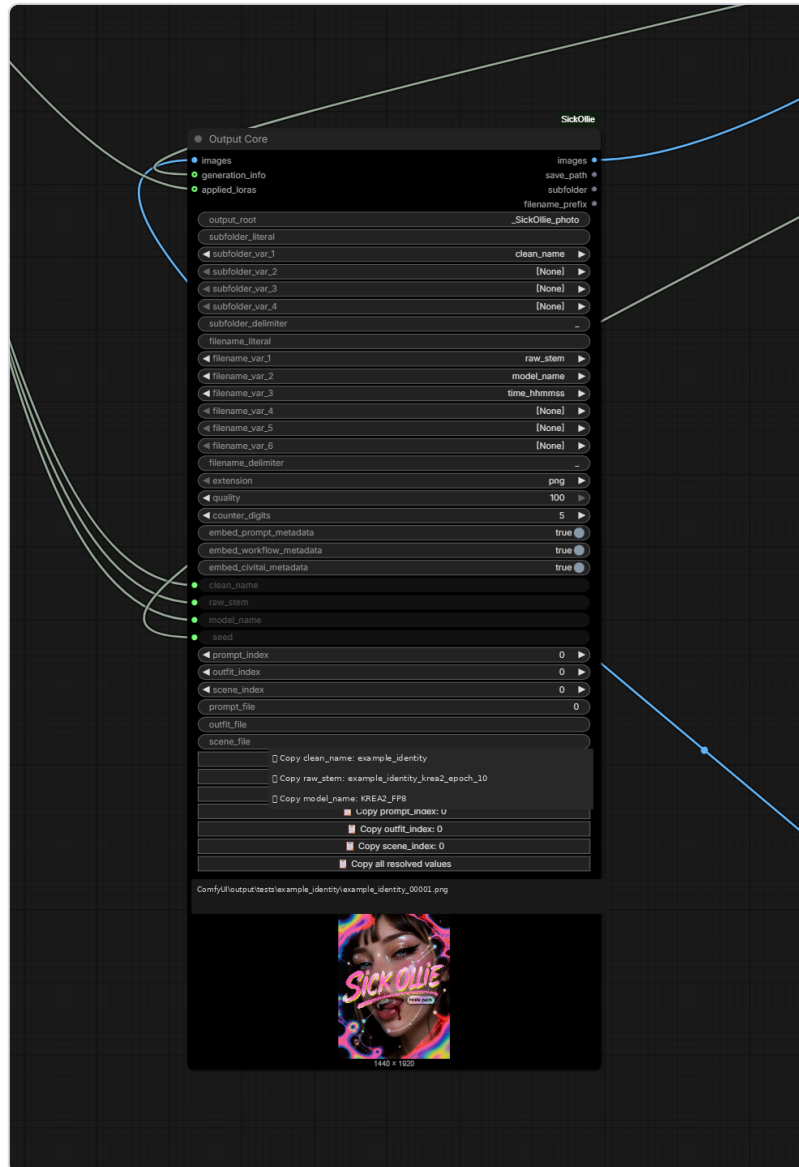
Reproducibility: Copy the resolved seed together with the same model, LoRA stack, prompt, dimensions, steps, sampler, scheduler, CFG, denoise, and shift.

Outputs

Output	Description
samples	Sampled latent batch for VAE Decode.
vae	Loaded VAE object.
seed_used	Exact resolved seed.
width / height	Final dimensions after custom or preset resolution calculation.
generation_info	JSON-like production record consumed by Output Core for metadata.

Chapter 4: Output Core

Output Core is a metadata-aware file builder. It creates portable subfolders and filenames from selected variables, saves PNG, JPG, or WebP images, and preserves enough production information for later inspection and Civitai resource matching.



Output Core with variable builders, metadata switches, connected resolved values, copy buttons, and the latest saved path.

Folder and Filename Construction

The subfolder builder combines one literal, up to four variables, and a delimiter. The filename builder combines one literal, up to six variables, and its own delimiter. Empty parts and [None] are omitted.


```

output_root: _SickOllie_tests
subfolder_literal:
subfolder_var_1: clean_name

filename_var_1: raw_stem
filename_var_2: model_name
filename_var_3: time_hhmmss
filename_delimiter: _

Result:
ComfyUI/output/_SickOllie_tests/example_identity/
example_identity_krea2_epoch_10_KREA2_FP8_221530_00001.png

```

Variable families

Available variables include resolved names, log indexes and files, seed, date and time fields, counters, and model-related values. The dropdowns prevent spelling mistakes and keep naming recipes portable.

Recommended split: Use `clean_name` for human-readable folders and `raw_stem` for exact versioned filenames.

Metadata and Civitai Resource Detection

Output Core combines three information streams:

- Comfy prompt data and workflow JSON.
- Generation Core's exact prompt, seed, sampler, scheduler, dimensions, denoise, CFG, shift, VAE, and other settings.
- Loader Core's exact applied-LoRA list.

When Civitai metadata is enabled, Output Core hashes the detected base model, VAE, and applied LoRA files and writes Civitai-style parameter fields. A local hash cache avoids recalculating unchanged resources every time.

Visible metadata switches

Visible label	Purpose
embed_prompt_metadata	Embed Comfy prompt data inside the saved file.
embed_workflow_metadata	Embed workflow data so supported images can reload the graph.
embed_civitai_metadata	Write Civitai-readable generation fields and resource hashes.

Only applied resources are recorded: Loader Core reports LoRAs that actually affected the model. A selected but disabled main or secondary LoRA is omitted from `applied_loras` and should not appear as an active Civitai resource.

Resolved-Value Tools

Output Core displays copy buttons for `clean_name`, `raw_stem`, `model_name`, prompt index, outfit index, and scene index, plus one button that copies all resolved values. The latest values and saved path survive workflow reloads.

Output Core Controls

`images`

Image tensor to save.

`generation_info`

Optional Generation Core production record.

`applied_loras`

Optional Loader Core list of active LoRAs and strengths.

`output_root`

Root folder relative to ComfyUI/output.

`subfolder_literal`

Fixed text added to the subfolder builder.

`subfolder_var_1..4`

Dropdown-selected subfolder variables.

`subfolder_delimiter`

Join text for subfolder parts.

`filename_literal`

Fixed text added to the filename builder.

`filename_var_1..6`

Dropdown-selected filename variables.

`filename_delimiter`

Join text for filename parts.

`extension`

png, jpg, or webp.

`quality`

JPG/WebP quality; PNG ignores it.

`counter_digits`

Width of the numeric file counter.

`embed_* metadata`

Prompt, workflow, and Civitai metadata switches.

`clean_name / raw_stem / model_name`

Connected or manual naming values.

`seed / prompt_index / outfit_index /`

`scene_index`

Connected or manual numeric values.

`prompt_file / outfit_file / scene_file`

Optional source-log filenames.

`saved_path`

Read-only most recent output path.

Output Core Outputs

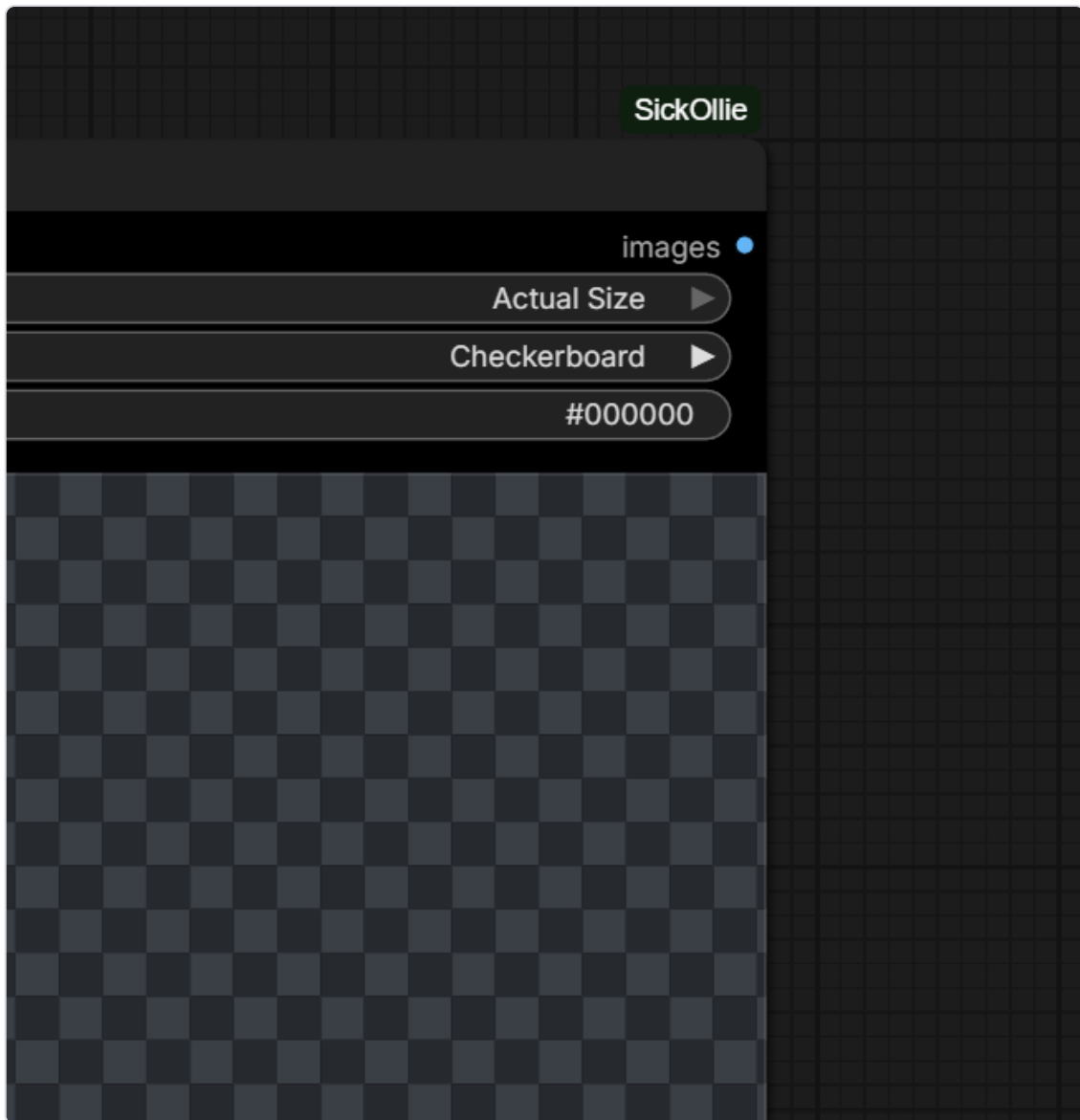
Output	Description
<code>images</code>	Original image tensor passed through unchanged.
<code>save_path</code>	Full path to the most recently saved file.
<code>subfolder</code>	Resolved subfolder segment.
<code>filename_prefix</code>	Resolved filename prefix before Comfy counter and extension.

Chapter 5: Preview Core

Preview Core is a resizable display node. It changes only how an image is presented on the canvas. The original image tensor passes through unchanged.

Fit Modes

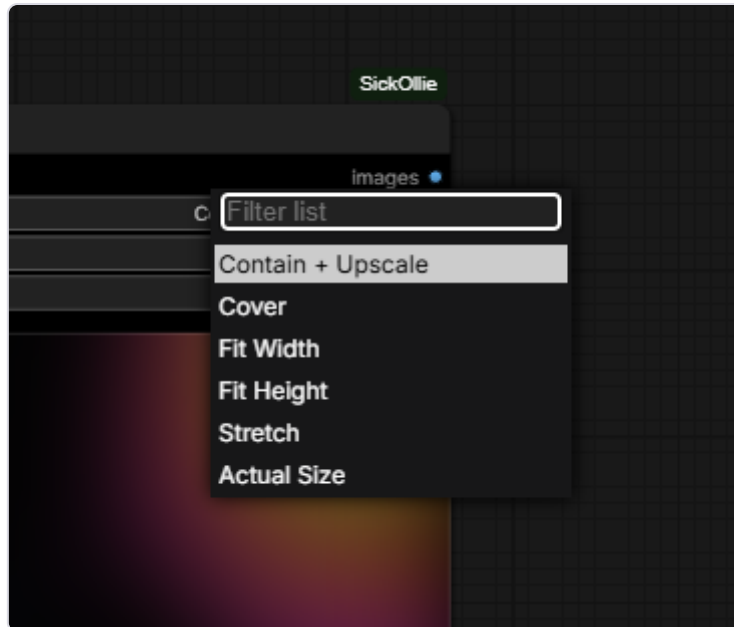
Mode	Display behavior
Contain + Upscale	Shows the full image and allows enlargement to fill available space.
Cover	Fills the preview area while preserving proportions, cropping edges when necessary.
Fit Width	Matches the preview width.
Fit Height	Matches the preview height.
Stretch	Fills the exact preview dimensions without preserving aspect ratio.
Actual Size	Displays the image at its natural pixel dimensions when possible.



Fit modes control only the canvas presentation.

Background Modes

Mode	Behavior
Solid	Uses the configured hex background color.
Checkerboard	Displays transparency-friendly checkerboard tiles.
Blurred Image	Builds a softened enlargement of the image behind the fitted foreground.



Blurred Image creates a gallery-like surround for tall or wide images.

Preview Persistence

The latest preview image reference is stored inside workflow properties. When the graph is rebuilt by switching tabs, reopening a workflow, or importing a saved PNG, Preview Core restores the image instead of returning to an empty waiting state.

Controls and Output

Field	Description
<code>images</code>	Image tensor to preview.
<code>fit_mode</code>	Contain, cover, width, height, stretch, or actual size.
<code>background_mode</code>	Solid, checkerboard, or blurred image.
<code>background_color</code>	Hex value used by Solid mode.
<code>images</code> output	Original image tensor passed through unchanged.

Chapter 6: Persist + Display Resolved Prompt

This optional utility is a string passthrough with a read-only display. It is useful when a final prompt should remain visible in a compact dedicated node or when the prompt must be branched to another destination.

Controls

Field	Meaning
<code>text</code>	Resolved string input, commonly Prompt Core <code>final_prompt</code> .
<code>saved_prompt</code>	Read-only display restored from saved workflow properties.

Output

`text` returns the original string unchanged.

Optional in v1.1: Prompt Core now preserves its own resolved display reliably. Keep this node when a separate compact display or additional text branch is useful.

Complete Wiring Guide

Essential production wiring

```

Loader Core model
  → Generation Core model

Prompt Core final_prompt
  → Generation Core positive_text

Generation Core samples
  → VAE Decode samples
Generation Core vae
  → VAE Decode vae

VAE Decode IMAGE
  → Output Core images
Output Core images
  → Preview Core images

```

Identity and naming wiring

```

Loader Core clean_name
  → Prompt Core name_value
Loader Core clean_name
  → Output Core clean_name
Loader Core raw_stem
  → Output Core raw_stem
Loader Core diffusion_model_stem
  → Output Core model_name
Loader Core applied_loras
  → Output Core applied_loras

```

Automatic trigger prefix

```

Loader Core main_trigger
  → Prompt Core prefix_text

Prompt Core prefix_enabled = true

```

The trigger is read from the currently selected SafeTensor for immediate display. During execution, the socket remains blank when the primary LoRA is inactive.

Exact generation metadata

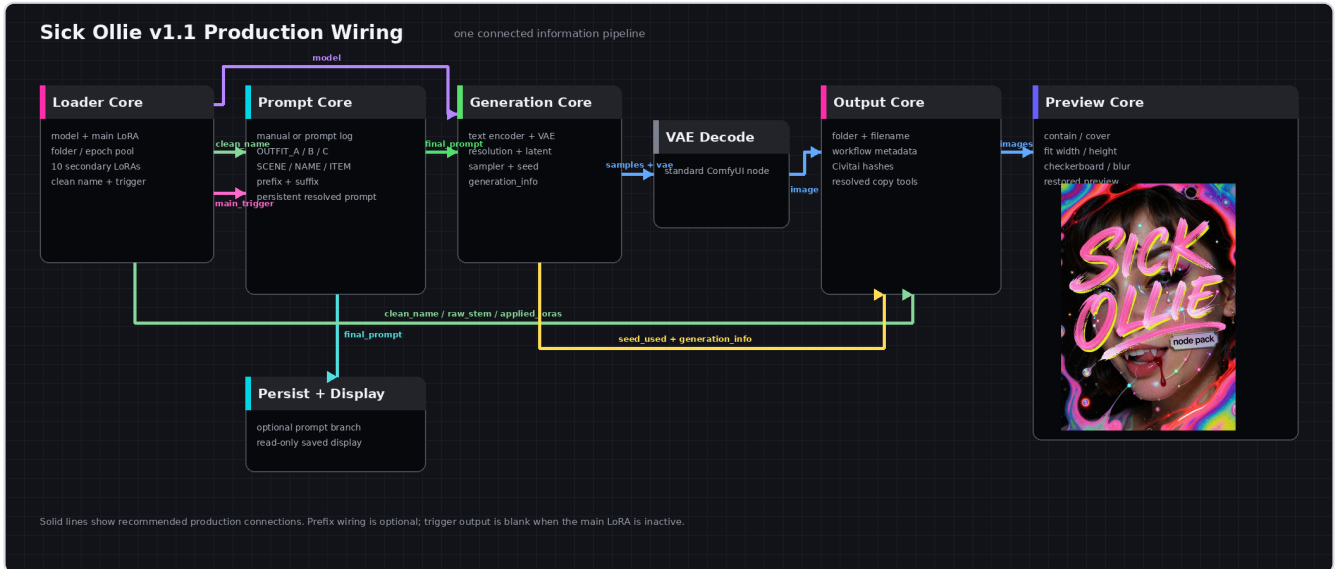
```

Generation Core seed_used
  → Output Core seed
Generation Core generation_info
  → Output Core generation_info

```


Optional prompt-log naming

```
Prompt Core prompt_index_resolved → Output Core prompt_index
Prompt Core outfit_index_resolved → Output Core outfit_index
Prompt Core scene_index_resolved → Output Core scene_index
Prompt Core prompt_file → Output Core prompt_file
Prompt Core outfit_file → Output Core outfit_file
Prompt Core scene_file → Output Core scene_file
```



Reference v1.1 wiring with main_trigger feeding Prompt Core prefix_text.

Workflow Recipes

Recipe 1: Test every epoch of one identity

1. Select the identity's folder.
2. Choose one value from `epoch_filter` or leave `[All epochs]`.
3. Set progression to increment and enable looping.
4. Use `clean_name` for the output folder and `raw_stem` for the filename.
5. Queue the number of images required.

Each queued prompt receives the next eligible main LoRA while the final files retain exact epoch stems.

Recipe 2: One identity, randomized prompt and outfit

```
Loader clean_name → Prompt name_value
Prompt source = log
prompt_mode = randomize
outfit_mode_A = randomize
scene_mode = randomize
```

Each log loops automatically. Blank lines are ignored.

Recipe 3: Three subjects, three outfits

```
Prompt template:
NAME poses with two friends. The first wears OUTFIT_A,
the second wears OUTFIT_B, and the third wears OUTFIT_C.

Assign a separate outfit file and mode to A, B, and C.
```

Recipe 4: Persistent style stack plus cycling identities

1. Use the main folder pool for identities.
2. Add style, lighting, color, and texture LoRAs as secondary rows.
3. Keep secondary rows enabled while the main LoRA progresses.
4. Save `applied_loras` through Output Core.

Secondary rows now survive tab changes and workflow reloads, so the stack does not need to be rebuilt every session.

Recipe 5: Trigger only when required

1. Connect `main_trigger` to Prompt Core `prefix_text`.
2. Enable the prefix.
3. Leave the connection in place for mixed LoRA libraries.

Triggerless LoRAs contribute a blank string. Disabled LoRAs also contribute a blank execution output, so the separator does not create a stray prompt fragment.

Recipe 6: Human folders, forensic filenames

```
Output subfolder_var_1 = clean_name
Output filename_var_1  = raw_stem
Output filename_var_2  = model_name
Output filename_var_3  = seed
```

The folder stays readable while the filename preserves the exact training version and generation seed.

Troubleshooting Guide

The main-LoRA dropdown shows unrelated files

Confirm the intended folder is selected. Disable `include_subfolders` when only direct children should appear. Check the epoch filter for `[All epochs]` or an unintended epoch.

The main LoRA changes when I select a folder

Changing the pool causes Loader Core to choose a valid value from the new pool when the previous selection is no longer eligible. This is expected.

The primary cycle keeps producing None

Enable `skip_none_during_cycle`. None remains manually selectable but is removed from automatic progression.

The selected main LoRA does not affect the model

- Confirm `main_enabled` is on.
- Confirm main strength is nonzero.
- Confirm the selected file is compatible with the loaded diffusion model.
- Check Loader Core `applied_loras` after execution.

The trigger button says none

The SafeTensor may not contain an explicit trigger, usable tag-frequency candidate, or model title. Many Krea2 character LoRAs work without a prompt trigger. Loader Core intentionally does not guess from the filename.

The trigger button shows a value but the connected prefix is blank

The main LoRA is probably disabled, None, or zero strength. The immediate button reports selected-file metadata for reference; the execution output is active-only by design.

Secondary LoRAs disappeared

Version 1.1 stores ten persistent slots. Confirm the workflow is using the v1.1 Loader Core JavaScript and Python files, then restart ComfyUI and hard-refresh. Rows from the current version should survive workflow reloads.

The Add Secondary LoRA button stops responding

All ten slots may be occupied. Right-click a row and choose Clear Row to free a slot.

clean_name still includes training suffixes

Open the clean-name dropdown and select a more aggressive trim level. The available choices depend on the selected filename and shared suffixes in the active pool.

NAME, OUTFIT_A, or SCENE remains in the prompt

- Matching is case-sensitive.
- Confirm the configured token exactly matches the prompt text.
- Confirm the relevant value or log line is not blank.
- Confirm the selected log file is available beneath SickOllieLogs.

OUTFIT_B or OUTFIT_C appears to have no public output socket

The final prompt contains all three replacements. Public outfit line/index/count/file outputs remain a primary compatibility set representing A, with B/C fallback. Inspect the resolved prompt or embedded workflow metadata for the complete result.

Prompt values become scrambled after reopening

Confirm both `prompt_core.py` and `js/prompt_core.js` come from the same v1.1 release. Restart and hard-refresh after replacement. Current v1.1 workflows preserve control order and resolved values.

The resolved prompt disappears after switching tabs

Current Prompt Core stores the latest result in workflow properties. Confirm the image or workflow was saved after executing the current node version.

New log files do not appear

Place UTF-8 text files beneath `ComfyUI/input/SickOllieLogs/prompts`, `outfits`, or `scenes`. Restart ComfyUI and hard-refresh.

Output Core creates an unexpected filename

Review the literal, variable dropdowns, delimiter, and whether connected input values are blank. Use the resolved-value copy buttons to inspect the exact naming inputs.

Output metadata is missing a LoRA

Connect Loader Core `applied_loras` to Output Core. Disabled, None, and zero-strength resources are intentionally omitted.

Civitai does not recognize a resource

The local file hash must match a Civitai-hosted file. Private, renamed, modified, or unpublished resources may remain unmatched even though their local hash is correctly embedded.

Preview Core says Waiting for image

Confirm Output Core images is connected and the upstream graph has executed. A restored preview requires a workflow saved after Preview Core received an image.

A saved PNG does not reload the workflow

Use PNG and enable `embed_workflow_metadata`. Some external editors strip metadata when resaving images.

Glossary

Term	Definition
Active LoRA	A LoRA that is enabled, selected, nonzero strength, and actually applied.
Applied LoRAs	The newline-separated list of LoRAs that affected the model, including strengths.
Clean Name	Human-friendly name produced by Loader Core's selected trim level.
Epoch Filter	Folder-pool restriction built from epoch markers detected in LoRA filenames.
Main LoRA	Primary LoRA controlled by folder filtering and queue-time progression.
Main Trigger	SafeTensor trigger detected from explicit metadata, tag frequency, or model title, emitted only while the main LoRA is active.
Prompt Log	One reusable prompt per nonblank line in a UTF-8 text file.
Raw Index	Editable line index before wrapping into the current log length.
Resolved Index	Actual valid line position after wrapping.
Raw Stem	Exact filename without parent folder or extension.
Secondary LoRA	Persistent fixed LoRA applied after the main LoRA and excluded from primary cycling.
Token	Literal case-sensitive placeholder such as NAME, SCENE, or OUTFIT_A.
Workflow Persistence	Node properties and serialized values that allow visible state to survive workflow reconstruction.

Compatibility, Credits, and License

Compatibility and release scope

- Public node IDs remain unchanged for Loader Core, Prompt Core, Generation Core, Output Core, Preview Core, and Persist + Display Resolved Prompt.
- Existing v1 workflows are supported through compatibility handling in the updated nodes.
- rgthree-comfy remains required for Loader Core's secondary-LoRA interface.
- The pack targets current ComfyUI frontend behavior and should be restarted and hard-refreshed after updates.

Credits

Created for the Sick Ollie ComfyUI production workflow. Loader Core's enhanced secondary-LoRA interactions adapt portions of rgthree-comfy's MIT-licensed interface utilities. See [THIRD_PARTY_NOTICES.md](#) in the release package.

License

See the repository's [LICENSE](#) file for the current license terms.